Fachhochschule Nordwestschweiz
Hochschule für Technik

Input
Regularisation, Data Augmentation

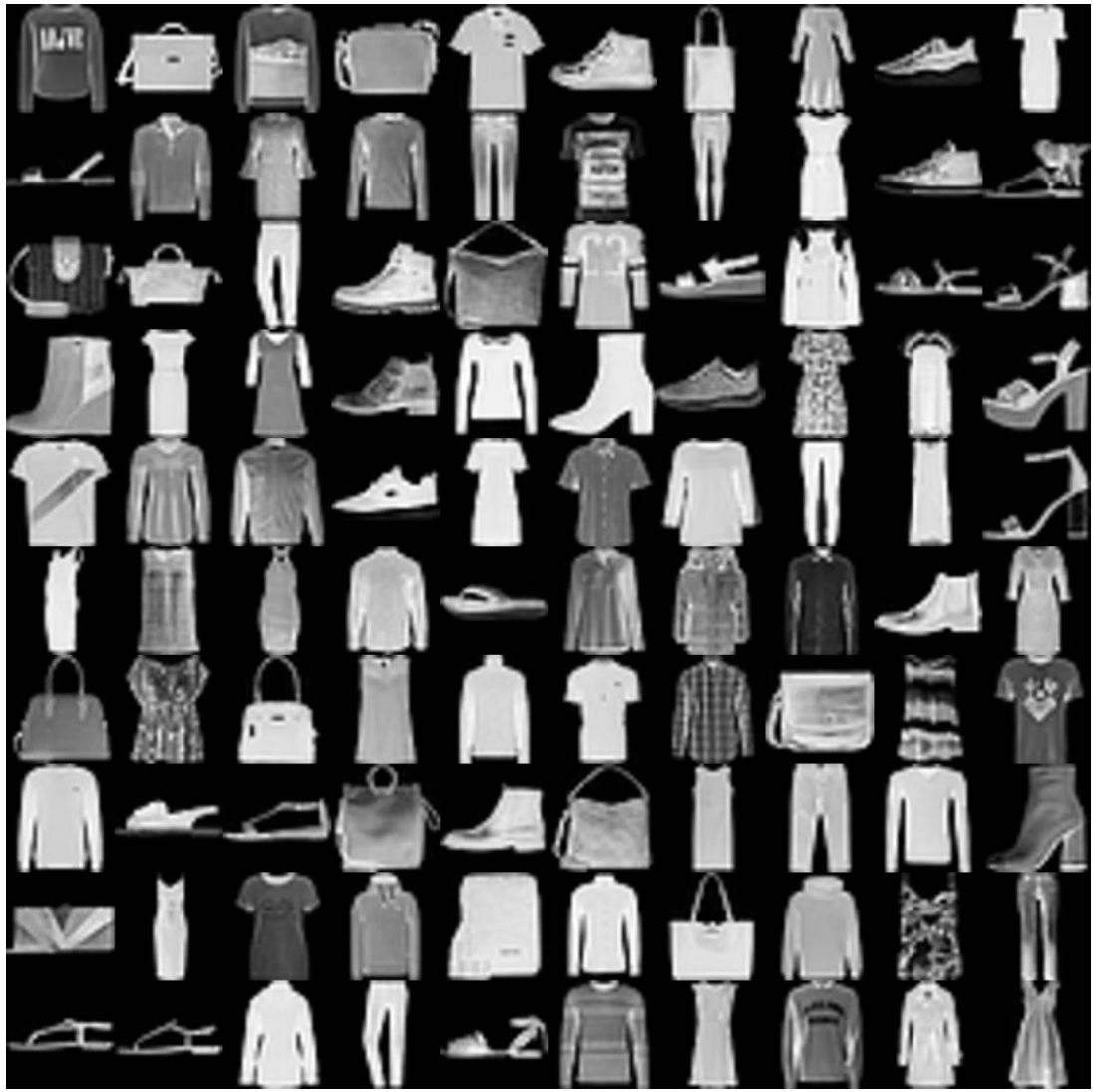
Deep Learning

HS 2025

Martin Melchior

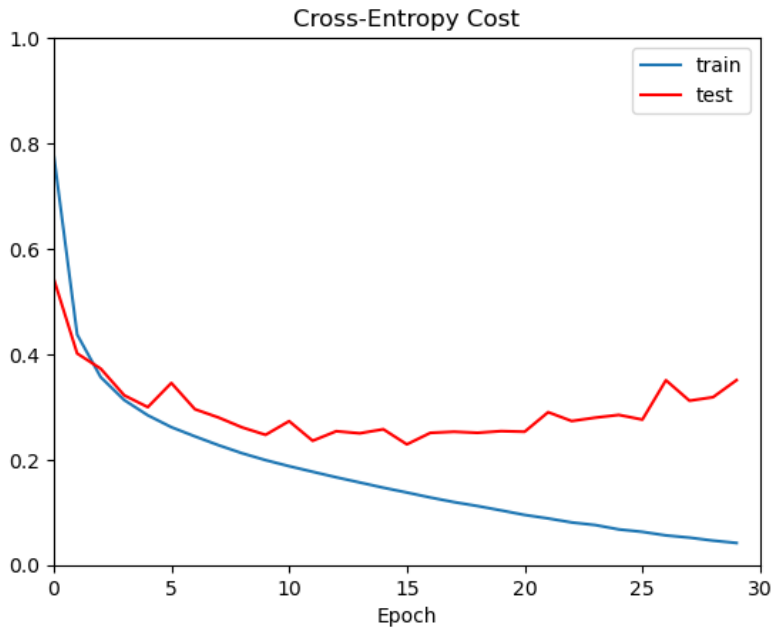
FashionMNIST

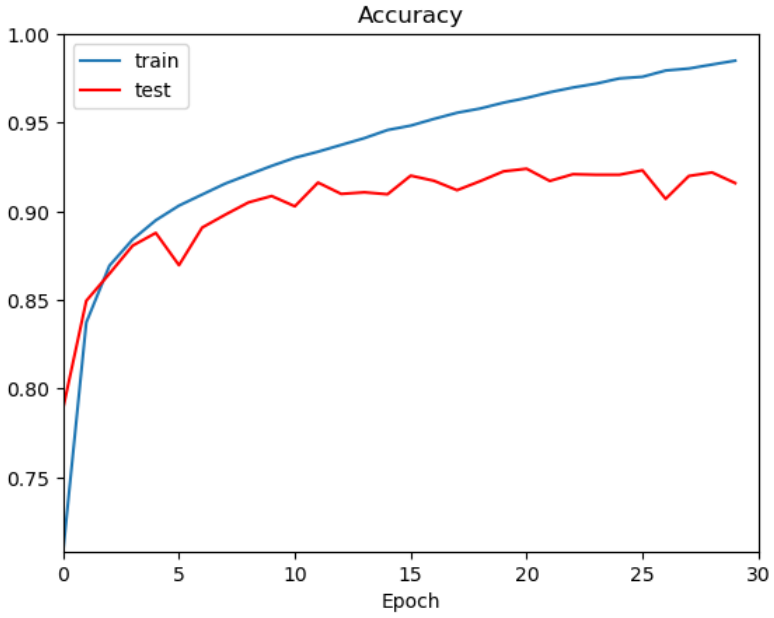
Same structure as MNIST,
but different conten:
clothes, boots, bags



Overfitting Model

```
class CNN2Layer3MLPLayer(torch.nn.Module):

    def __init__(self, nhidden1=240, nhidden2=80, noutput=10):
        super(CNN2Layer3MLPLayer, self).__init__()
        self.layers = torch.nn.Sequential(
            torch.nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1, bias=True),
            torch.nn.ReLU(),
            torch.nn.Conv2d(in_channels=32, out_channels=32, kernel_size=3, padding=1, bias=True),
            torch.nn.ReLU(),
            torch.nn.MaxPool2d(kernel_size=3, stride=2, padding=1),
            torch.nn.Flatten(),
            torch.nn.Linear(14*14*32, nhidden1),
            torch.nn.ReLU(),
            torch.nn.Linear(nhidden1, nhidden2),
            torch.nn.ReLU(),
            torch.nn.Linear(nhidden2, noutput)
        )

    def forward(self, x):
        z = self.layers(x)
        return z
```



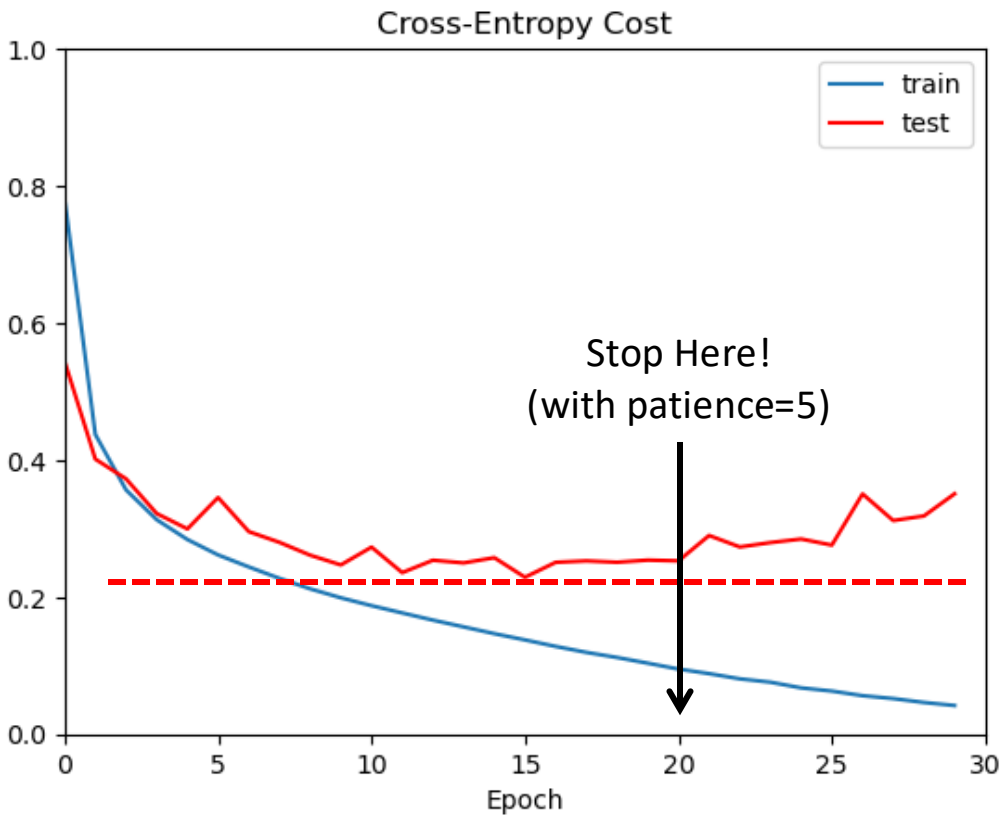


Overfitting can be caused by having networks with too many parameters that are trained on too few samples. Through training, the model learns *by heart* and generalises poorly.

Regularisation Methods

- Early Stopping
- L1 / L2 Regularisation
- Dropout – specifically for NNs
- Increase Training Dataset -> Data Augmentation

Early Stopping



Possible Implementation (Source: Stackoverflow)

```
class EarlyStopper:
    def __init__(self, patience=1, min_delta=0):
        self.patience = patience
        self.min_delta = min_delta
        self.counter = 0
        self.min_validation_loss = float('inf')

    def early_stop(self, validation_loss):
        if validation_loss < self.min_validation_loss:
            self.min_validation_loss = validation_loss
            self.counter = 0
        elif validation_loss > (self.min_validation_loss + self.min_delta):
            self.counter += 1
            if self.counter >= self.patience:
                return True
        return False
```

See also this blog.

See GML

L1 / L2 - Regularisation

This would include also the bias terms and possibly parameters from layer that you don't want to regularize (e.g. Batchnorm).

```
def norms_params(model):
    l1_norm = 0.0
    l2_norm = 0.0
    for p in model.parameters():
        l1_norm += torch.norm(p,1)
        l2_norm += torch.norm(p,2)**2
    return l1_norm, l2_norm
```

Here, we consider using a version that is restricted to weights and which also allows you to control the layer types for which to regularize the weights (e.g. without batchnorm)

```
def norms_weights(model):
    l1_norm = 0.0
    l2_norm = 0.0
    for m in model.modules():
        if isinstance(m, (torch.nn.Linear, torch.nn.Conv2d)):
            l1_norm += torch.norm(m.weight,1)
            l2_norm += torch.norm(m.weight,2)**2
    return l1_norm, l2_norm

l1_all, l2_all = norms_params(model)
l1_weights, l2_weights = norms_weights(model)
print(l1_all, l2_all)
print(l1_weights, l2_weights)
```

```
tensor(10492.4170, grad_fn=<AddBackward0>) tensor(132.2547, g
tensor(10481.7666, grad_fn=<AddBackward0>) tensor(130.8874, g
```

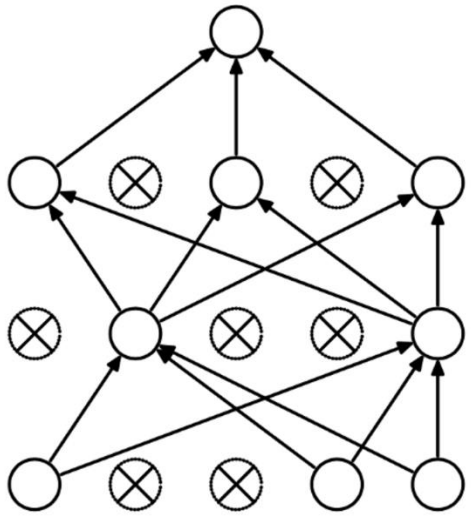
L2-Reg also integrated in optimisers as `weight_decay`-parameter.

Dropout

- Performance Boost:
  - State-of-the-art neural networks got a 1–2% accuracy boost simply by adding dropout (at the time it was introduced).
- Algorithm:
  - *Training time*: At each training step, each neuron in a layer (*excl. output layer*) has a probability *p* (“dropout rate”) of being ignored during this step
    —> activations masked
  - *Inference time*: For testing or in production, neurons don’t get dropped, but weights or outputs need to be corrected since weights are trained including the dropout rate.
- Default dropout rate:
  - 50% for hidden layer
  - 20% for input layer

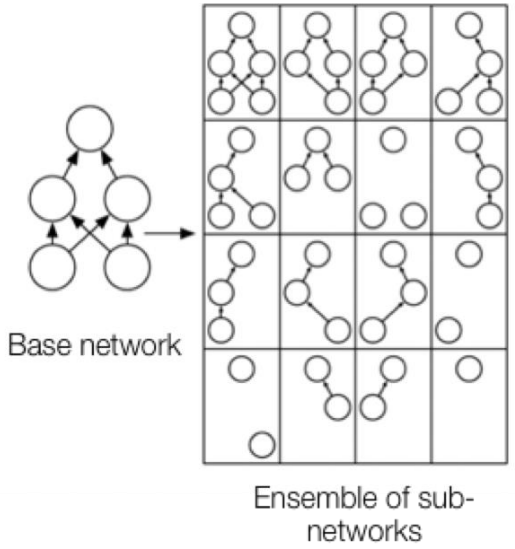
Proposed by G. Hinton et al. in 2012, “Improving neural networks by preventing co-adaptation of feature detectors”

Further detailed in a paper by N. Srivastava et al. in 2014, “Dropout: A Simple Way to Prevent Neural Networks from Overfitting”.



Intuition on Why Dropout Works

- Implication of using dropout:
  - Simpler models with less units are considered during training.
  - Models learn to be less dependent on single units and units cannot co-adapt with their neighbouring units; they have to be as useful as possible on their own.
  - A more robust network is obtained that generalises better.



From Geron, “Hands-On Machine Learning with Scikit-Learn and TensorFlow”:
“Would a company perform better if its employees were told to toss a coin every morning to decide whether or not to go to work? Well, who knows; perhaps it would! The company would obviously be forced to adapt its organization; it could not rely on any single person to fill in the coffee machine or perform any other critical tasks, so this expertise would have to be spread across several people. Employees would have to learn to cooperate with many of their coworkers, not just a handful of them. The company would become much more resilient. If one person quit, it wouldn’t make much of a difference. “

Implementation of Dropout

- Computationally very cheap:
  - For each unit only one random binary number needs to be drawn: O(n) computations per update step (mini-batch)
  - O(n) additional memory to store these binary numbers for the backprop.
  - No additional cost at test time or in production.

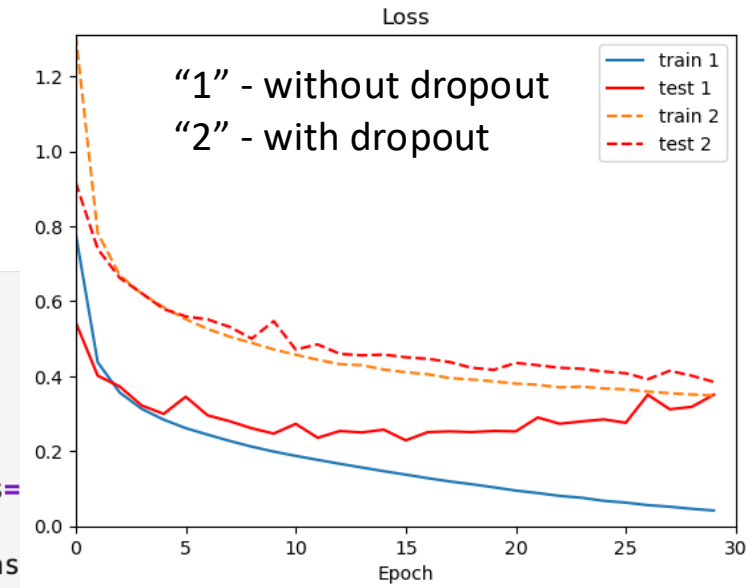
- Reduced representational capacity:
  - Dropout reduces the effective capacity of a model -> capacity may need to be increased which may be more expensive to be trained.

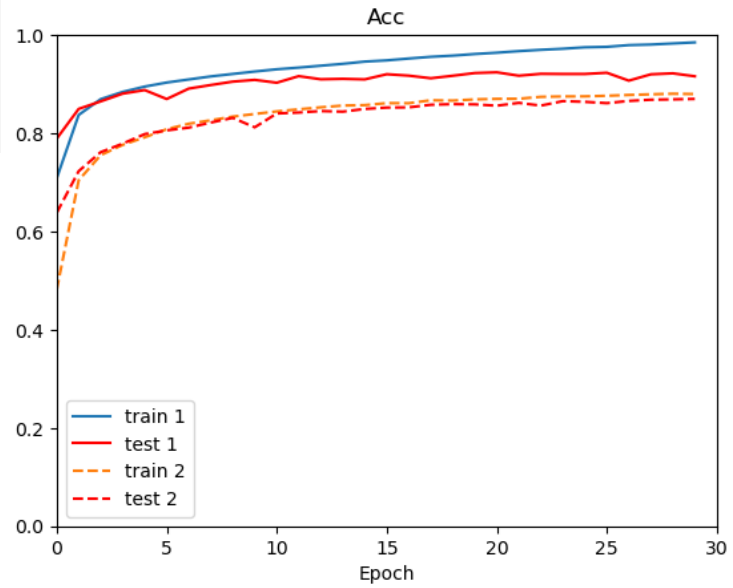
- Very versatile:
  - Does not significantly limit the type of model or training procedure that can be used (applicable e.g. to MLP, CNN or RNNs or other optimisers).
  - Can be combined with other regularisation techniques.

Implementation in PyTorch

```python
class CNN2Layer3MLPLayerDropout(torch.nn.Module):

    def __init__(self, nhidden1=240, nhidden2=80, dropout_rate=0.7, noutput=10):
        super(CNN2Layer3MLPLayerDropout, self).__init__()
        self.layers = torch.nn.Sequential(
            torch.nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1, bias=
            torch.nn.ReLU(),
            torch.nn.Conv2d(in_channels=32, out_channels=32, kernel_size=3, padding=1, bias
            torch.nn.ReLU(),
            torch.nn.MaxPool2d(kernel_size=3, stride=2, padding=1),
            torch.nn.Flatten(),
            torch.nn.Dropout(p=dropout_rate),
            torch.nn.Linear(14*14*32, nhidden1),
            torch.nn.ReLU(),
            torch.nn.Dropout(p=dropout_rate),
            torch.nn.Linear(nhidden1, nhidden2),
            torch.nn.ReLU(),
            torch.nn.Dropout(p=dropout_rate),
            torch.nn.Linear(nhidden2, noutput)
        )

    def forward(self, x):
        z = self.layers(x)
        return z
```

```
----------------------------------------------------------------
        Layer (type)               Output Shape         Param #
================================================================
            Conv2d-1           [-1, 32, 28, 28]             320
              ReLU-2           [-1, 32, 28, 28]               0
            Conv2d-3           [-1, 32, 28, 28]           9,248
              ReLU-4           [-1, 32, 28, 28]               0
         MaxPool2d-5           [-1, 32, 14, 14]               0
           Flatten-6                 [-1, 6272]               0
           Dropout-7                 [-1, 6272]               0
            Linear-8                  [-1, 240]       1,505,520
              ReLU-9                  [-1, 240]               0
          Dropout-10                  [-1, 240]               0
           Linear-11                   [-1, 80]          19,280
             ReLU-12                   [-1, 80]               0
          Dropout-13                   [-1, 80]               0
           Linear-14                   [-1, 10]             810
================================================================
Total params: 1,535,178
Trainable params: 1,535,178
Non-trainable params: 0
----------------------------------------------------------------
```





Dropout for Conv-Layers

- Use `torch.nn.Dropout2d`

```
dropout_layer = torch.nn.Dropout2d(p=0.5)
x = torch.randn(1, 3, 4, 4)
x_dropped = dropout_layer(x)
```

input x

```
tensor([[[[-0.8292, -1.2145,  1.2466,  0.6935],
          [-0.8694,  1.1506, -1.6004, -0.7610],
          [ 0.0290,  0.1859,  0.4726, -0.6716],
          [ 1.3418,  0.7296, -0.3023,  1.3315]],

         [[-1.0526,  2.0200, -0.2726,  0.3111],
          [ 0.0539,  1.9058, -0.6082,  0.3496],
          [ 1.6153, -1.8453,  0.6224,  0.8125],
          [-0.0907,  1.2793,  0.6297, -0.4314]],

         [[-0.9393, -1.1878,  0.1708, -0.0026],
          [-0.4356, -0.1869, -1.5094, -1.8166],
          [-1.3008,  0.4521,  1.0328,  1.6839],
          [-0.7217, -0.5702, -0.2268, -0.3707]]]])
```

Dropout2d

```
tensor([[[[-0.0000, -0.0000,  0.0000,  0.0000],
          [-0.0000,  0.0000, -0.0000, -0.0000],
          [ 0.0000,  0.0000,  0.0000, -0.0000],
          [ 0.0000,  0.0000, -0.0000,  0.0000]],

         [[-0.0000,  0.0000, -0.0000,  0.0000],
          [ 0.0000,  0.0000, -0.0000,  0.0000],
          [ 0.0000, -0.0000,  0.0000,  0.0000],
          [-0.0000,  0.0000,  0.0000, -0.0000]],

         [[-1.8786, -2.3756,  0.3415, -0.0052],
          [-0.8712, -0.3738, -3.0188, -3.6333],
          [-2.6015,  0.9041,  2.0657,  3.3678],
          [-1.4433, -1.1405, -0.4537, -0.7414]]]])
```

Dropout

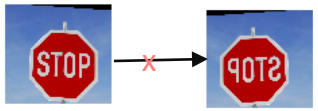
```
tensor([[[[-1.6584, -2.4291,  2.4932,  0.0000],
          [-1.7389,  2.3012, -0.0000, -0.0000],
          [ 0.0000,  0.3718,  0.9452, -1.3431],
          [ 0.0000,  0.0000, -0.6047,  0.0000]],

         [[-2.1052,  4.0401, -0.5453,  0.6222],
          [ 0.0000,  0.0000, -0.0000,  0.6992],
          [ 3.2306, -0.0000,  0.0000,  0.0000],
          [-0.0000,  0.0000,  0.0000, -0.8628]],

         [[-0.0000, -0.0000,  0.3415, -0.0000],
          [-0.8712, -0.0000, -0.0000, -3.6333],
          [-2.6015,  0.0000,  2.0657,  3.3678],
          [-1.4433, -0.0000, -0.4537, -0.7414]]]])
```

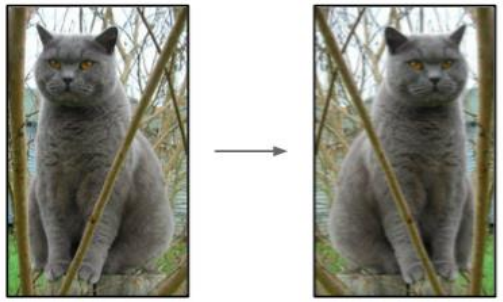
Data Augmentation

2D data augmentation

- Quantity of “perturbations” are randomly chosen

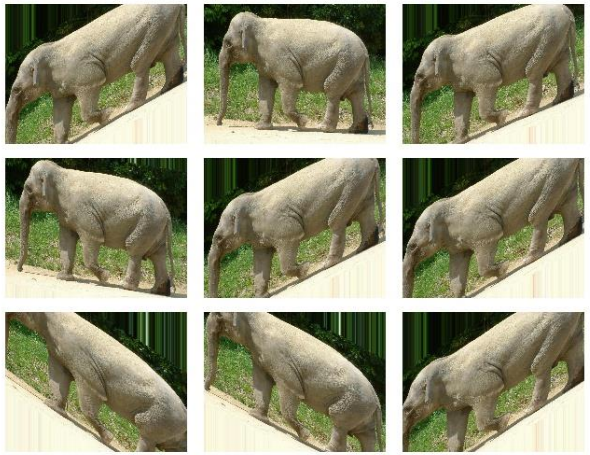
- Rotation
  - range of degrees
- Translation
  - percentage of shift in a range
  - for width and height
- Flip
  - may not be applicable to all types of images
  - e.g. images with text



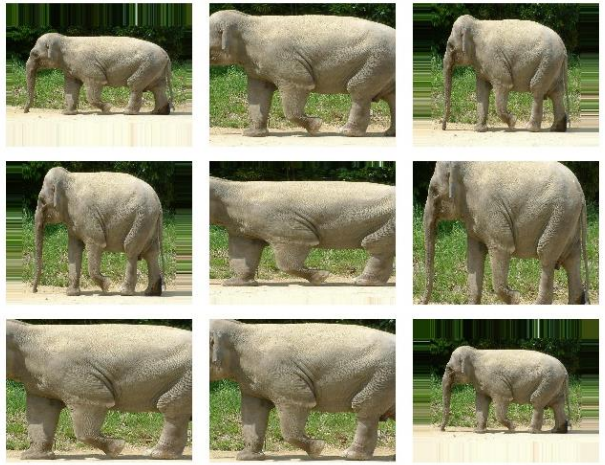


2D data augmentation

- Shear



- Zoom, Crop and Reshape

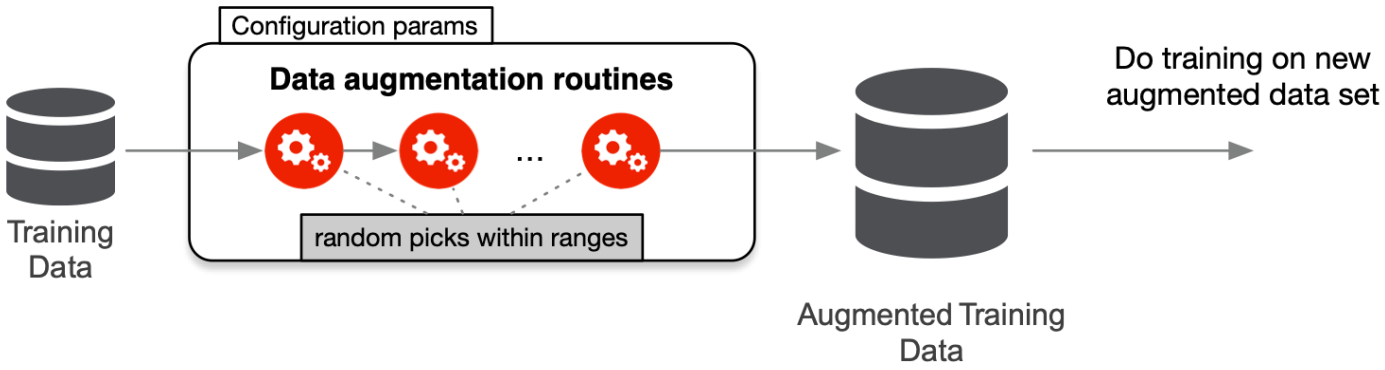


- Color
  - shifting
  - illumination
- …

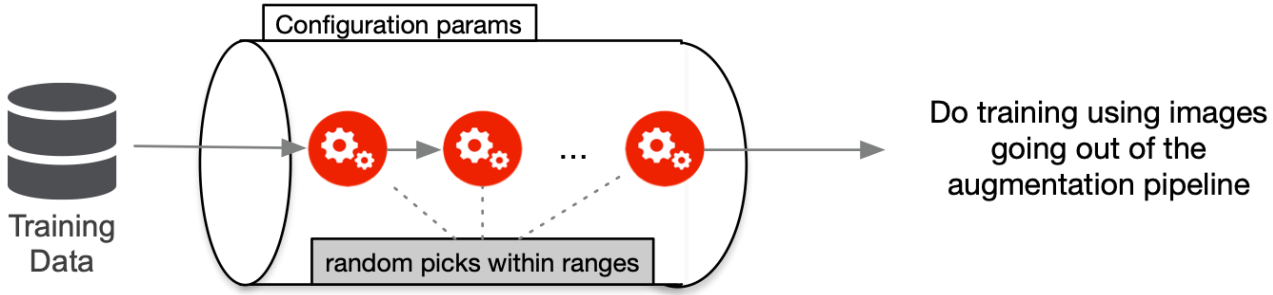
STOP

Data augmentation - implementation strategies

Data augmentation - pre-augmentation



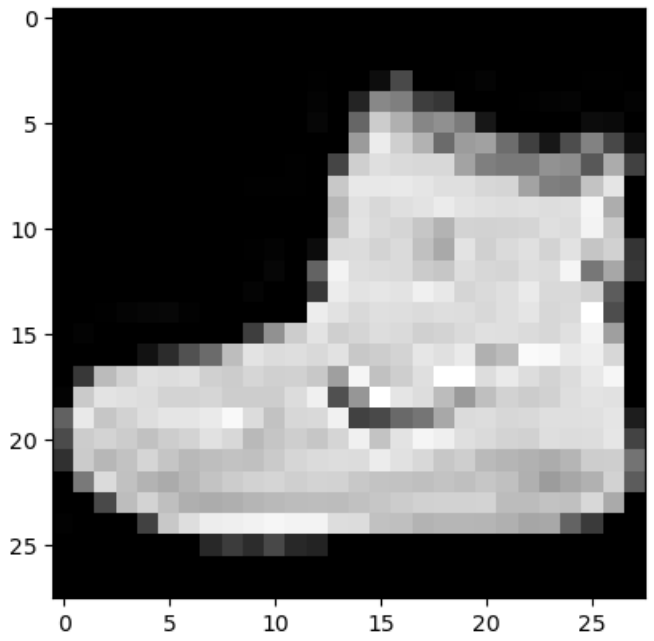
Data augmentation - online augmentation

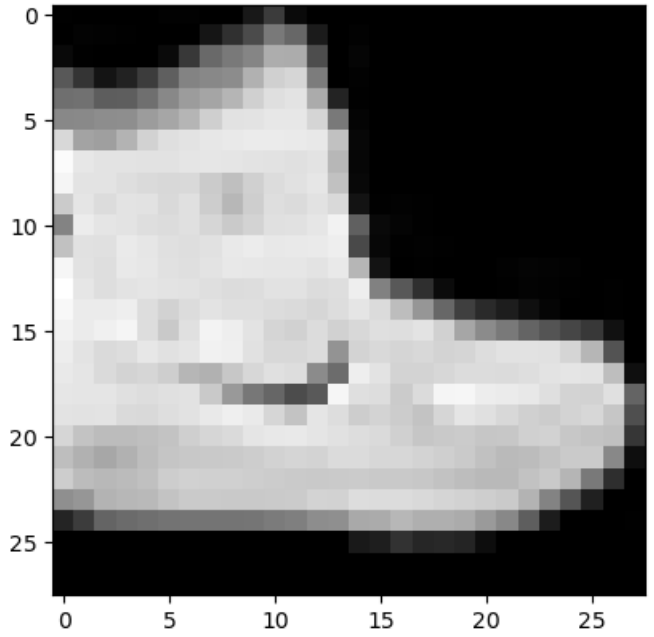


Implementation PyTorch (online)

```
from torchvision.transforms import v2

transforms = v2.Compose([
    v2.RandomResizedCrop(size=(28, 28), scale = (0.7, 1.0)),
    v2.RandomHorizontalFlip(p=0.5),
    v2.ToTensor(),
])
```

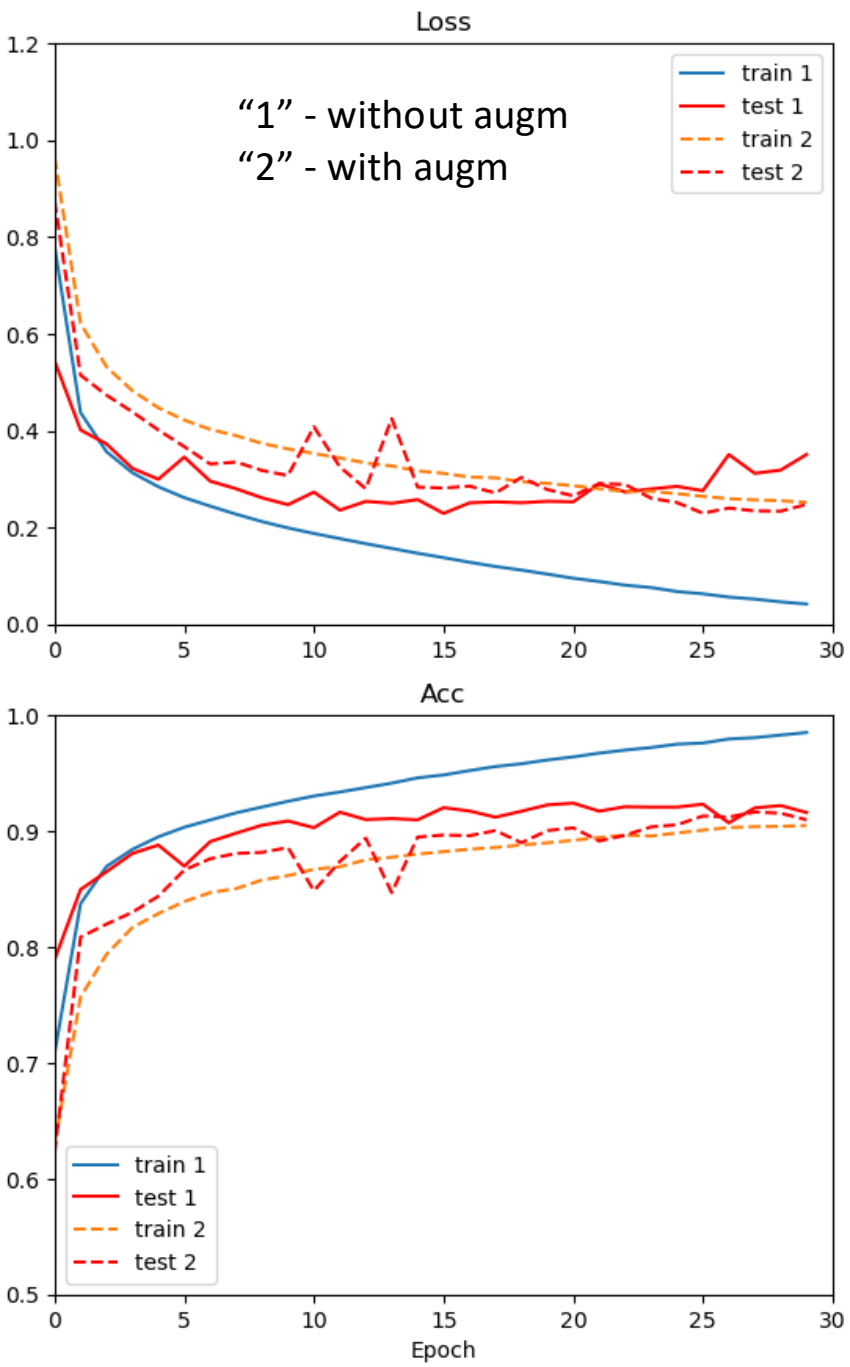
```
img, label = training_data[0]
img_trsf = transforms(img)
plt.figure(1)
plt.imshow(img.squeeze(), cmap="gray")
plt.figure(2)
plt.imshow(img_trsf.squeeze(), cmap="gray")
plt.show()
```





Implementation PyTorch (online)

```
training_data = datasets.FashionMNIST(
    root="../data",
    train=True,
    download=True,
    transform=transforms
)
test_data = datasets.FashionMNIST(
    root="../data",
    train=False,
    download=True,
    transform=ToTensor()
)
```



Expected Risk Minimisation – Bias-Variance Trade-off

Loss and Risk / Cost

- The loss L measures how good (or bad) a model f performs on a given sample, i.e. is a mapping

$$f, (x, y) \to L(f, (x, y)) \geq 0$$

larger values mean worse performance

Classification: $L(f, (x, y)) = -\log(f_y(x))$ ……

y-th component $f_y(x)$ of $f(x)$ which is *K*-dim,

- Observed data generated by *probability distribution*

$$Z = (X, Y) \sim P(\cdot)$$

includes labels

- The *statistical / expected risk* is defined by

$$R(f) = \mathbb{E}_{Z \sim P(\cdot)}\left[L(f, Z)\right]$$

expectation over Z distributed with P(); unknown since P() is unknown.

From Expected to Empirical Risk Minimisation

Expected Risk Minimisation

$$f_{\text{opt}} = \underset{f}{\operatorname{argmin}} \mathbb{E}_{Z\sim P}[L(f, Z)]$$

not tractable - P unknown

use empirical distribution D_N with N (training) samples $z^{(i)} \sim P$

$$f_{D_N} = \underset{f}{\operatorname{argmin}} \mathbb{E}_{Z\sim D_N}[L(f, Z)]$$

not useful - memorise samples

use parametrised family of models f_θ , suited to learn the patterns in the data

Empirical Risk Minimisation

$$f_{\hat{\theta}(D_N)} = \underset{\theta}{\operatorname{argmin}} \frac{1}{N} \sum_{i=1}^{N} \left[L(f_\theta, z^{(i)})\right]$$

depends on selected (training) samples, family of models.

Properties of the Minimum Empirical Risk Estimator

$$f_{\hat{\theta}(D_N)} = \underset{\theta, D_N \text{fixed}}{\operatorname{argmin}} \frac{1}{N} \sum_{z \in D_N} [L(f_\theta, z)] = \underset{\theta(D_N \text{fixed})}{\operatorname{argmin}} \mathbb{E}_{Z \sim D_N}[L(f_\theta, Z)]$$

- How does it depend on the considered class of parametric models f_θ?
- What is a good choice of parametric models given the problem at hand and the data available?

- How does it vary with the specifically selected training set D_N?
- How can we make it more robust against the specifically selected training set D_N?

Bias-Variance Decomposition

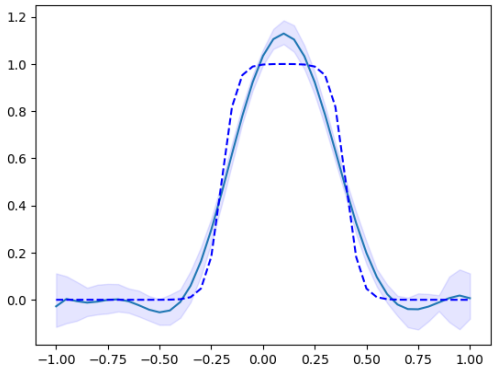
point estimate averaged over differently sampled training sets and training runs

arbitrary fixed (x, y), not seen in training

$$\mathbb{E}_{\mathcal{D}}\left[\left(f_{\hat{\theta}\,(D)}(x) - y\right)^2\right]$$

$$= \mathbb{E}_{\mathcal{D}}\left[\left(f_{\hat{\theta}\,(D)}(x) - \mathbb{E}_{\mathcal{D}}\left[f_{\hat{\theta}\,(D)}(x)\right] + \mathbb{E}_{\mathcal{D}}\left[f_{\hat{\theta}\,(D)}(x)\right] - y\right)^2\right]$$

$$= \underbrace{\mathbb{E}_{\mathcal{D}}\left[\left(f_{\hat{\theta}\,(D)}(x) - \mathbb{E}_{\mathcal{D}}\left[f_{\hat{\theta}\,(D)}(x)\right]\right)^2\right]} + \underbrace{\left(\mathbb{E}_{\mathcal{D}}\left[f_{\hat{\theta}\,(D)}(x)\right] - y\right)^2}$$



$\mathrm{Var}_{\mathcal{D}}\left[f_{\hat{\theta}\,(D)}(x)\right]$ **Variance**

in plot :
~ grey-shaded area

Bias

may also include a noise term (noisy data) which is irreducible

in plot:
~difference between continuous and dashed line

Bias-Variance Estimated in Practice

$$\underbrace{\mathbb{E}_{D\in\mathcal{D}}\left[\left(f_{\hat{\theta}(D)}(x)-y\right)^2\right]}_{\textbf{Error}} = \underbrace{\left(\mathbb{E}_{D\in\mathcal{D}}\left[f_{\hat{\theta}\,(D)}(x)\right]-y\right)^2}_{\textbf{Bias}} + \underbrace{\mathrm{Var}_{D\in\mathcal{D}}\left[f_{\hat{\theta}\,(D)}(x)\right]}_{\textbf{Variance}}$$

Error

as rough proxy often taken as the test/validation error

$$\mathbb{E}_{D_{\text{test}}}\left[\left(f_{\hat{\theta}(D_N)}(X)-Y\right)^2\right]$$

one single training based on D_N, averaged over test / validation set

D_{test}(disjoint from D_N)

Bias

as rough proxy often used the training error

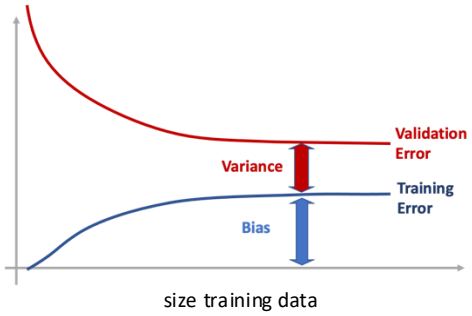
$$\left(\mathbb{E}_{D_N}\left[f_{\hat{\theta}(D_N)}(X)-Y\right]\right)^2$$

one single training based on D_N, averaged over training set

training set should not be too small

Variance

as rough proxy often used the difference between test/validation and training error

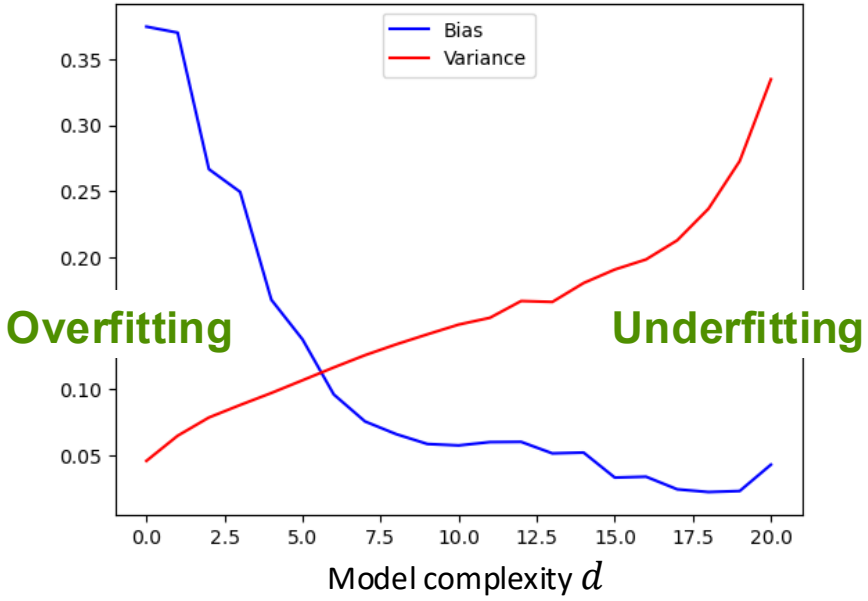


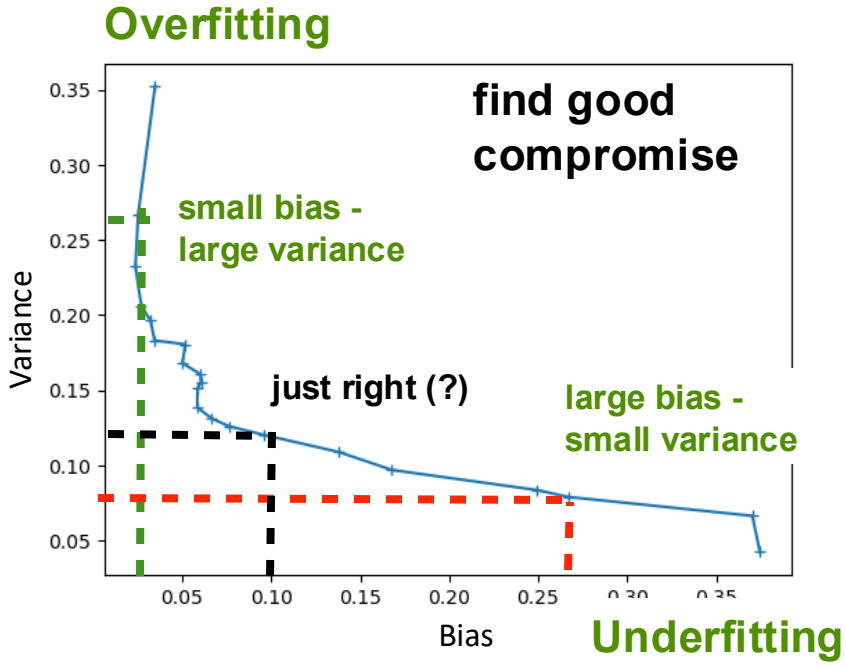
Bias-Variance Tradeoff

For a toy example:

Bias: Discrepancy between model predictions averaged over training runs.

Variance: Variation of model predictions around averaged model prediction.

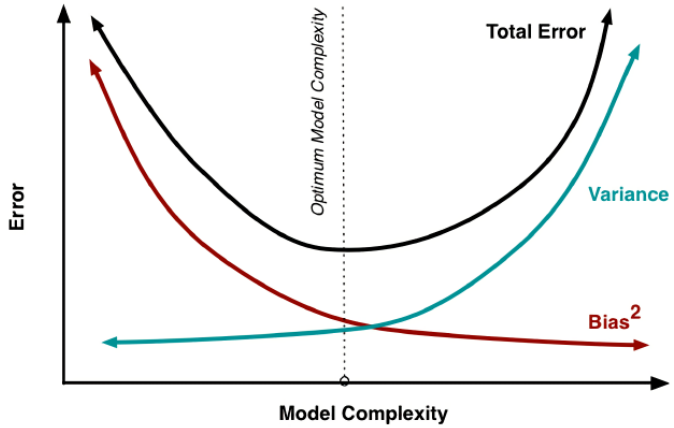




Deep Double Descent

Pre-Deep Learning Era

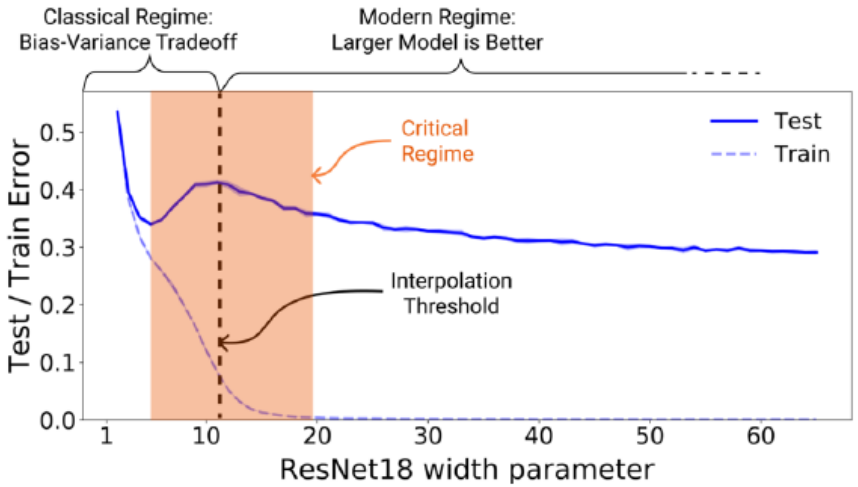
Bias-Variance-Tradeoff seen as guiding principle to select model complexity, number of epochs, etc.



S.Geman, et al.: Neural networks and the bias/variance dilemma. Neural Computation, 4(1):1–58, 1992.

Modern Deep Learning Era

Bias and Variance as two components that can be simultaneously improved.

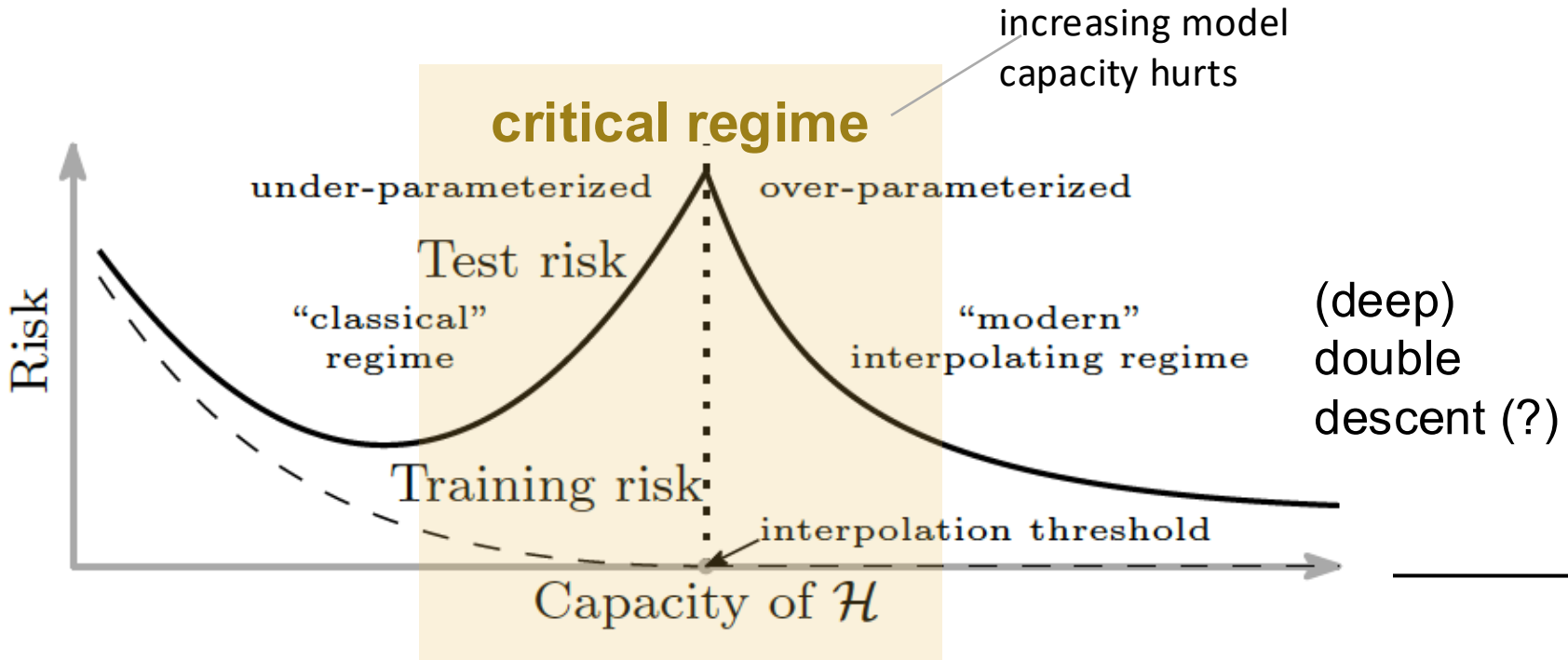


P.Nakkiran: https://arxiv.org/abs/1912.02292 (2019)
R.Schaeffer: https://arxiv.org/pdf/2303.14151.pdf (2024)
B.Neil: https://arxiv.org/abs/1810.08591 (2018)
M.Belkin: https://arxiv.org/abs/1812.11118 (2018)

How to Choose Model Capacity?

Evaluate the performance on the training and the test set

- for different models (with different model complexity)
- by using different training set sizes (more training data, less overfitting)
- after different number of training epochs



(deep) double descent (?)

Guidance in Practical Applications

- Successful for applications of DNNs for unstructured data (e.g. CV, language, sound); i.e. when working with unstructured data, DNNs provide a good *inductive bias* (e.g. CNNs, Transformers).

- Compare with *human performance.*
- Deep double descent observed to occur in various cases.
- Try to minimise Bias and Variance independently!
  There is not necessarily a tradeoff.

- Consider methods to oppose overfitting!